Cleaning and Visualising Data

Missing Values, Scatter Plots, and Interpretation

Name: _____ Date: _____

The data

Instead of the 15-row example matrix, the assignment uses the file *export_english.csv*: weekly COVID-19 case counts for 2020–2022, 8,753 rows, split by health region, area, sex, age group and socioeconomic index. Each row also gives the population of that group. Every observation is stored as **[x, y] = [population, cases]**. A third column, **socioeconomic_index**, is kept as well, to show that `clean_matrix` works for any number of columns.

The file has no empty cells, but it does have missing values, coded in two ways. These are turned into Python missing values when the file is loaded:

Raw value in the CSV	Meaning	Loaded as	Rows
socioeconomic_index = -1	the source's code for "unknown"	None	94
population = 0 (with cases > 0)	unknown group (sex/age "Not available")	NaN	11
empty cell	blank entry	""	0

Task 1: Clean the matrix

`is_missing(v)` returns True for `None`, for float NaN (checked with `math.isnan` after `None` and strings are ruled out), and for empty or whitespace-only strings. `clean_matrix(matrix, fill=-1)` deep-copies the input, loops over every cell, swaps each missing value for `fill`, and returns the new matrix together with the list of (row, column) positions it replaced. The original matrix is left unchanged, and an assertion in the script checks this. The function was also run on part of the toy matrix from the assignment sheet.

Program output (only part of the matrix is printed; all 8,753 rows would not fit):

```
Matrix size: 8753 rows x 3 columns

Original matrix (first 5 rows + first 10 rows with missing values):
    row | population | cases | socio_idx
      0 |      10667 |     4 |         4
      1 |      18507 |     3 |         3
      2 |      17456 |     1 |         3
      3 |       3747 |     1 |         4
      4 |       6454 |     1 |         4
     66 |        nan |     1 |      None
     70 |        nan |     1 |      None
    100 |        nan |     1 |      None
    145 |        984 |     1 |      None
    175 |        574 |     1 |      None
    195 |        568 |     1 |      None
    247 |       4256 |     1 |      None
    255 |        nan |     1 |      None
    271 |        nan |     1 |      None
    286 |        nan |     1 |      None

Cleaned matrix (same rows):
    row | population | cases | socio_idx
      0 |      10667 |     4 |         4
      1 |      18507 |     3 |         3
      2 |      17456 |     1 |         3
      3 |       3747 |     1 |         4
      4 |       6454 |     1 |         4
     66 |         -1 |     1 |        -1
     70 |         -1 |     1 |        -1
    100 |         -1 |     1 |        -1
```

```
    145 |        984 |     1 |        -1
    175 |        574 |     1 |        -1
    195 |        568 |     1 |        -1
    247 |       4256 |     1 |        -1
    255 |         -1 |     1 |        -1
    271 |         -1 |     1 |        -1
    286 |         -1 |     1 |        -1

Values replaced: 105 (in 94 rows)
  population           11
  cases                0
  socioeconomic_index  94

Toy matrix check: 5 replaced -> [[1.0, 2.1], [3.0, -1], [-1, 10.1], [7.0, -1], [9.0, -1], [-1, 22.0]]
```

Result: 105 values were replaced, in 94 rows: 94 socioeconomic index values and 11 population values. No case counts were missing.

Task 2: Plot the data

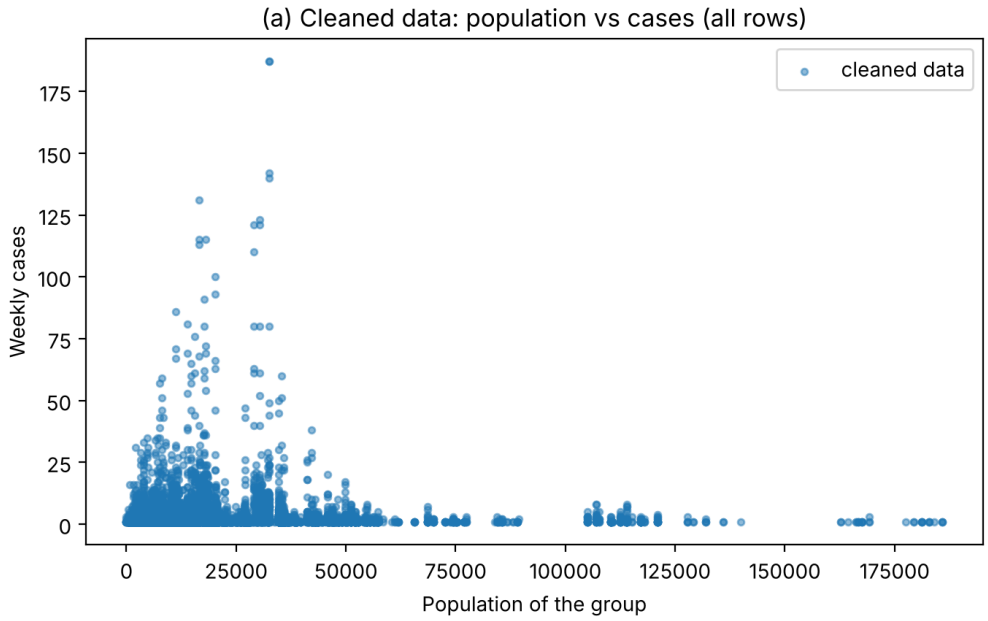


Figure 1 (a). Cleaned data: every row, including the replaced ones.

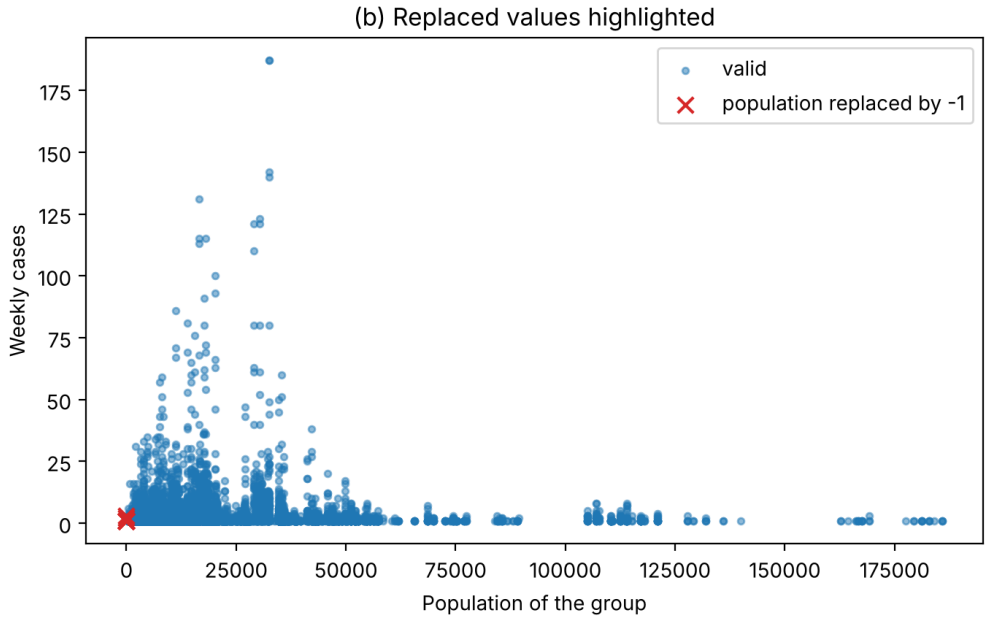


Figure 2 (b). The 11 rows whose population was replaced by -1 are shown as red crosses at x = -1.

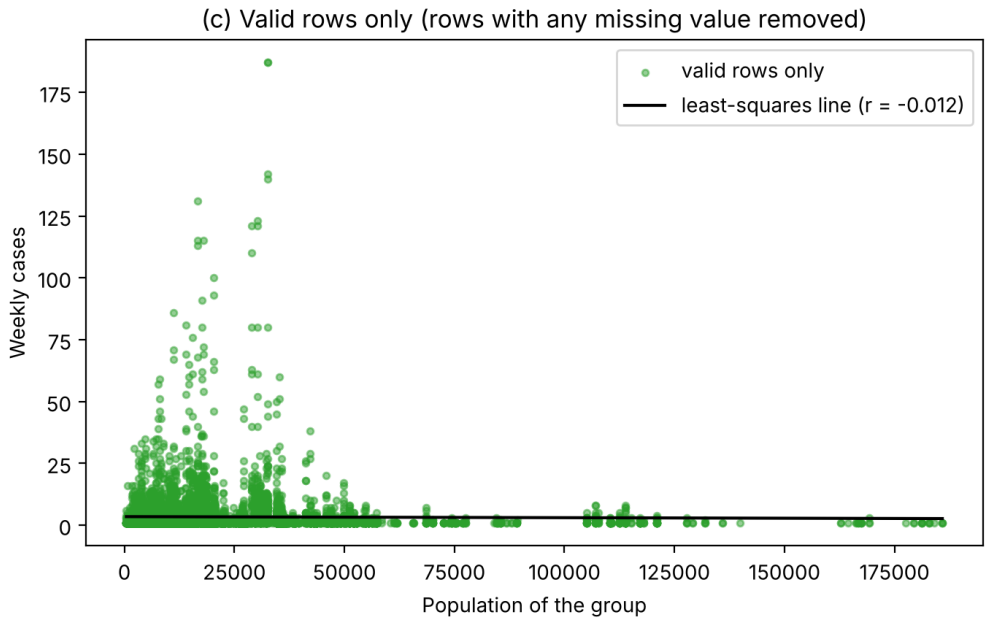


Figure 3 (c). Valid rows only: the 94 rows with any missing value are removed. A least-squares line is added.

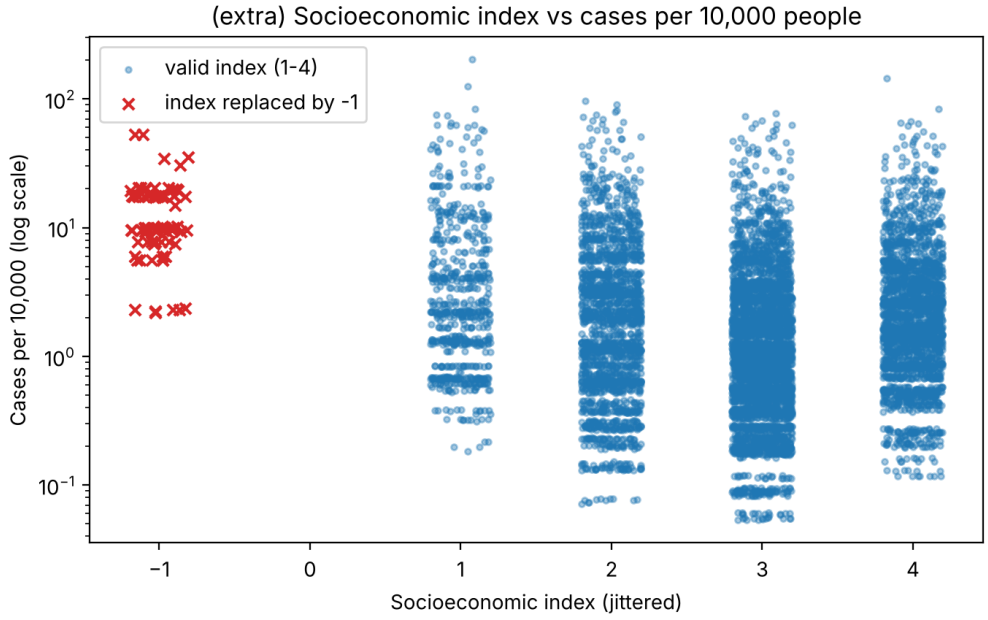


Figure 4 (extra). Socioeconomic index against cases per 10,000 people. Here the -1 placeholder sits on the x-axis as if it were a real index level.

Task 3: Describe the behaviour

(a) Trend. Population and weekly cases show no clear relationship. Most groups report only 1–5 cases a week, whatever their size, and the fitted line is almost flat (slope = -4.2×10^{-6}, r = -0.012; Figure 3). It is neither increasing nor decreasing, and not linear. The highest counts (up to 187) come from groups of 10,000–35,000 people.

(b) Replaced values. The 11 population placeholders lie at x = -1 (Figure 2). On a scale reaching 185,000 they blend in with the smallest real groups, so they look plausible. The socioeconomic placeholders stand out much more (Figure 4). They form an "index -1" column apart from levels 1–4, with rates about 10 per 10,000. The real levels have medians of 1–2 per 10,000.

(c) Distortion. -1 is treated as a real number. The mean socioeconomic index falls from 2.898 to 2.856. A trend line of rate against index would be pulled down by the high-rate cluster at -1 and suggest a steep, false pattern. A population of -1 also gives negative incidence rates, which make no sense. The population–cases line hardly changes (r = -0.009 against -0.012), only because 11 of the 8,753 rows are affected.

(d) Alternatives. Removing the affected rows costs only 94 rows (1.1%), so it is the safest choice here. Median imputation for population (13,602) keeps every row but invents values for groups whose size is truly unknown. For a categorical field such as the index, a separate "unknown" category is better than a number. A negative placeholder only suits formats that cannot store NaN, and must be filtered out before analysis.

Appendix: Python code (assignment.py)

```
"""
Data Handling with Python - Assignment: Cleaning and Visualising Data

The starting data is the COVID-19 case export (export_english.csv): one row
per week x area x sex x age group x socioeconomic index, with the number of
cases and the population of that group.

Each observation is used as [x, y] = [population, cases].  A third column,
socioeconomic_index, is also kept to show that clean_matrix works for any
number of columns.

Missing values in the raw file:
  * socioeconomic_index == -1  -> the source's own code for "unknown"  -> None
  * population == 0            -> impossible for a group that has cases -> NaN
  * any empty cell             -> ""   (handled, though none occur in this file)
"""

import copy
import csv
import math
import os

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt

HERE = os.path.dirname(os.path.abspath(__file__))
CSV_PATH = os.path.join(HERE, "data", "export_english.csv")
FIG_DIR = os.path.join(HERE, "figures")

nan = float("nan")


# ---------------------------------------------------------------------------
# Loading the data into a list of lists
# ---------------------------------------------------------------------------
def to_number(text):
    """Convert a CSV string to int/float; keep blanks as "" (missing)."""
    text = text.strip()
    if text == "":
        return ""
    try:
        return int(text)
    except ValueError:
        try:
            return float(text)
        except ValueError:
            return text


def load_data(path=CSV_PATH):
    """Return data = [[population, cases, socioeconomic_index], ...]."""
    data = []
    with open(path, newline="", encoding="utf-8") as f:
        for row in csv.DictReader(f, delimiter=";"):
            population = to_number(row["population"])
            cases = to_number(row["cases"])
            sei = to_number(row["socioeconomic_index"])

            if sei == -1:                       # source code for "unknown"
                sei = None
            if population == 0 and cases != 0:  # population can't be 0 here
                population = nan
            data.append([population, cases, sei])
    return data


# ---------------------------------------------------------------------------
# Task 1: clean the matrix
# ---------------------------------------------------------------------------
def is_missing(v):
    """True for None, NaN, and empty / whitespace-only strings."""
    if v is None:
```

```
        return True
    if isinstance(v, str):
        return v.strip() == ""
    if isinstance(v, float):
        return math.isnan(v)          # None was already excluded above
    return False


def clean_matrix(matrix, fill=-1):
    """Return (cleaned_copy, replaced_cells).

    Works for any number of rows and columns; the input is not modified.
    replaced_cells is a list of (row_index, col_index) pairs.
    """
    cleaned = copy.deepcopy(matrix)
    replaced = []
    for i, row in enumerate(cleaned):
        for j, value in enumerate(row):
            if is_missing(value):
                cleaned[i][j] = fill
                replaced.append((i, j))
    return cleaned, replaced


def fmt(v):
    if v is None:
        return "None"
    if isinstance(v, float) and math.isnan(v):
        return "nan"
    if v == "":
        return '""'
    return str(v)


def print_rows(title, matrix, rows):
    print(title)
    print(f"  {'row':>5} | {'population':>10} | {'cases':>5} | {'socio_idx':>9}")
    for i in rows:
        p, c, s = matrix[i]
        print(f"  {i:>5} | {fmt(p):>10} | {fmt(c):>5} | {fmt(s):>9}")


def linear_fit(xs, ys):
    """Least-squares slope, intercept and Pearson r (no numpy needed)."""
    n = len(xs)
    mx, my = sum(xs) / n, sum(ys) / n
    sxx = sum((x - mx) ** 2 for x in xs)
    syy = sum((y - my) ** 2 for y in ys)
    sxy = sum((x - mx) * (y - my) for x, y in zip(xs, ys))
    slope = sxy / sxx
    return slope, my - slope * mx, sxy / math.sqrt(sxx * syy)


# ---------------------------------------------------------------------------
# Task 2: plots
# ---------------------------------------------------------------------------
def make_plots(cleaned, bad_rows, original):
    os.makedirs(FIG_DIR, exist_ok=True)
    xs = [r[0] for r in cleaned]
    ys = [r[1] for r in cleaned]

    good = [i for i in range(len(cleaned)) if i not in bad_rows]
    bad = sorted(bad_rows)

    # (a) all cleaned points
    plt.figure(figsize=(7, 4.5))
    plt.scatter(xs, ys, s=10, alpha=0.5, color="tab:blue", label="cleaned data")
    plt.title("(a) Cleaned data: population vs cases (all rows)")
    plt.xlabel("Population of the group")
    plt.ylabel("Weekly cases")
    plt.legend()
    plt.tight_layout()
    plt.savefig(os.path.join(FIG_DIR, "plot1_cleaned.png"), dpi=200)
    plt.close()

    # (b) replaced population values highlighted
    pop_bad = [i for i in bad if is_missing(original[i][0])]
```

```
    plt.figure(figsize=(7, 4.5))
    plt.scatter([xs[i] for i in good], [ys[i] for i in good], s=10, alpha=0.5,
                color="tab:blue", label="valid")
    plt.scatter([xs[i] for i in pop_bad], [ys[i] for i in pop_bad], s=60,
                color="tab:red", marker="x", label="population replaced by -1")
    plt.title("(b) Replaced values highlighted")
    plt.xlabel("Population of the group")
    plt.ylabel("Weekly cases")
    plt.legend()
    plt.tight_layout()
    plt.savefig(os.path.join(FIG_DIR, "plot2_highlighted.png"), dpi=200)
    plt.close()

    # (c) only valid rows
    plt.figure(figsize=(7, 4.5))
    gx = [xs[i] for i in good]
    gy = [ys[i] for i in good]
    plt.scatter(gx, gy, s=10, alpha=0.5, color="tab:green", label="valid rows only")
    slope, icpt, r = linear_fit(gx, gy)
    lx = [min(gx), max(gx)]
    plt.plot(lx, [slope * x + icpt for x in lx], color="black", lw=1.5,
             label=f"least-squares line (r = {r:.3f})")
    plt.title("(c) Valid rows only (rows with any missing value removed)")
    plt.xlabel("Population of the group")
    plt.ylabel("Weekly cases")
    plt.legend()
    plt.tight_layout()
    plt.savefig(os.path.join(FIG_DIR, "plot3_valid_only.png"), dpi=200)
    plt.close()

    # (extra) socioeconomic index vs incidence: the -1 placeholder on the x-axis
    plt.figure(figsize=(7, 4.5))
    jitter = lambda i: ((i * 7919) % 100 - 50) / 250  # deterministic jitter
    gi = [i for i in range(len(cleaned)) if cleaned[i][0] > 0]
    rate = {i: cleaned[i][1] / cleaned[i][0] * 10_000 for i in gi}
    ok = [i for i in gi if not is_missing(original[i][2])]
    ph = [i for i in gi if is_missing(original[i][2])]
    plt.scatter([cleaned[i][2] + jitter(i) for i in ok], [rate[i] for i in ok],
                s=8, alpha=0.4, color="tab:blue", label="valid index (1-4)")
    plt.scatter([cleaned[i][2] + jitter(i) for i in ph], [rate[i] for i in ph],
                s=30, color="tab:red", marker="x", label="index replaced by -1")
    plt.yscale("log")
    plt.title("(extra) Socioeconomic index vs cases per 10,000 people")
    plt.xlabel("Socioeconomic index (jittered)")
    plt.ylabel("Cases per 10,000 (log scale)")
    plt.legend()
    plt.tight_layout()
    plt.savefig(os.path.join(FIG_DIR, "plot4_index_vs_rate.png"), dpi=200)
    plt.close()


# ---------------------------------------------------------------------------
# Main
# ---------------------------------------------------------------------------
def main():
    data = load_data()
    cleaned, replaced = clean_matrix(data, fill=-1)
    bad_rows = {i for i, _ in replaced}

    # Show the matrices (first rows + every row that contained a missing value;
    # printing all 8,753 rows would not be readable).
    sample = list(range(5)) + sorted(bad_rows)[:10]
    print(f"Matrix size: {len(data)} rows x {len(data[0])} columns\n")
    print_rows("Original matrix (first 5 rows + first 10 rows with missing values):",
               data, sample)
    print()
    print_rows("Cleaned matrix (same rows):", cleaned, sample)
    print()
    names = ["population", "cases", "socioeconomic_index"]
    print(f"Values replaced: {len(replaced)} (in {len(bad_rows)} rows)")
    for j, name in enumerate(names):
        print(f"  {name:<20} {sum(1 for _, c in replaced if c == j)}")

    # Original must be unchanged
    assert any(is_missing(v) for row in data for v in row)
    assert not any(is_missing(v) for row in cleaned for v in row)
    # Small check on the toy matrix from the assignment sheet
```

```
    toy = [[1.0, 2.1], [3.0, None], [None, 10.1], [7.0, nan], [9.0, ""], [nan, 22.0]]
    toy_clean, toy_rep = clean_matrix(toy)
    print(f"\nToy matrix check: {len(toy_rep)} replaced -> {toy_clean}")

    # Numbers for the Task 3 report
    xs = [r[0] for r in cleaned]
    ys = [r[1] for r in cleaned]
    good = [i for i in range(len(cleaned)) if i not in bad_rows]
    s_all, b_all, r_all = linear_fit(xs, ys)
    s_ok, b_ok, r_ok = linear_fit([xs[i] for i in good], [ys[i] for i in good])
    print("\nTrend population -> cases")
    print(f"  cleaned (with -1s): slope={s_all:.3e}  intercept={b_all:.3f}  r={r_all:.4f}")
    print(f"  valid rows only:    slope={s_ok:.3e}  intercept={b_ok:.3f}  r={r_ok:.4f}")

    sei_clean = [r[2] for r in cleaned]
    sei_valid = [r[2] for r in data if not is_missing(r[2])]
    print("\nMean socioeconomic index")
    print(f"  with -1 placeholders: {sum(sei_clean) / len(sei_clean):.3f}")
    print(f"  valid only:           {sum(sei_valid) / len(sei_valid):.3f}")
    pop_valid = sorted(r[0] for r in data if not is_missing(r[0]))
    print(f"  median population of valid rows: {pop_valid[len(pop_valid) // 2]}")

    make_plots(cleaned, bad_rows, data)
    print(f"\nFigures saved to {FIG_DIR}")


if __name__ == "__main__":
    main()
```